Suparna Dey Platform — System Architecture

> **Product:** Multi-tenant SaaS food-delivery platform **Model:** Hub & Spoke — one central control plane + N tenant forks **Authoring team:** Suparna Dey Platform Engineering **Doc version:** 1.0 · Last edited by ops handoff

Table of contents

1. Executive summary
2. High-level architecture
3. Hub — Suparna Dey Super Admin
4. Spoke — a tenant fork
5. The three apps per tenant
6. Cross-cutting concerns
7. Data model
8. Integration ecosystem
9. SaaS lifecycle
10. Deployment architecture
11. Security model
12. Scalability & operations
13. Roadmap

1 · Executive summary

Suparna Dey Platform is a multi-tenant SaaS that lets any restaurant launch a fully-featured food-delivery product in under 30 minutes. Each restaurant ("tenant") gets its own branded mobile apps + admin portal, its own MongoDB, and its own Emergent-hosted deployment — but they all report to and are billed by a single central hub (the "Suparna Dey Platform Super Admin").

Think of it as **Shopify for restaurant delivery**: one codebase (`Curry-Co-Template`), forked once per tenant, driven by two config files (`brand.json` + `brand.config.js`) and a shared control plane.

Architecture at a glance: - **Hub-and-spoke** — the hub owns Stripe + tenant registry + platform billing. Spokes (tenants) never see Stripe credentials. - **Fork-per-tenant** — every restaurant is its own GitHub repo, its own Emergent project, and its own Mongo database. Isolation is strong; noisy-neighbour blast radius is zero. - **Phone-home enforcement** — each tenant's backend polls the hub every 5 minutes; if the tenant's SaaS subscription lapses, customer checkout and driver logins are gated with a friendly 402. - **Three apps per tenant**, all built from one Expo codebase: customer mobile, delivery-partner mobile, admin web.

Phases 1, 2 and 3 are shipped. Phase 4 (platform-wide coupons + open integrations marketplace) is on the roadmap.

---

2 · High-level architecture

2.1 · The hub-and-spoke picture



2.2 · What the hub knows vs. what each tenant knows

Concern	Hub	Tenant
Stripe API keys		
Subscription plans (prices)		
Which tenants exist		
Customer orders		
Menu & catering data		
Delivery driver GPS		
Google Maps API key	(shared)	(allow-listed for this tenant)
SaaS subscription status	source of truth	cached, gated
Platform-wide coupons		consumed
Tenant brand assets		

2.3 · Why fork-per-tenant vs. multi-tenant single-DB

We considered two models:

Option	Chosen	Rationale
A — Fork-per-tenant	Yes	Full isolation. Tenant crashes / data loss / support customization stays scoped. Easier compliance (each tenant's data physically separate). Rebrand is a config-file swap, not a hosting-layer negotiation.
B — One codebase, tenant column on every row	No	Would need schema migrations to coordinate across every restaurant simultaneously. Single-tenant crashes bring down all restaurants. Rebrand becomes a runtime concern instead of a build-time one.

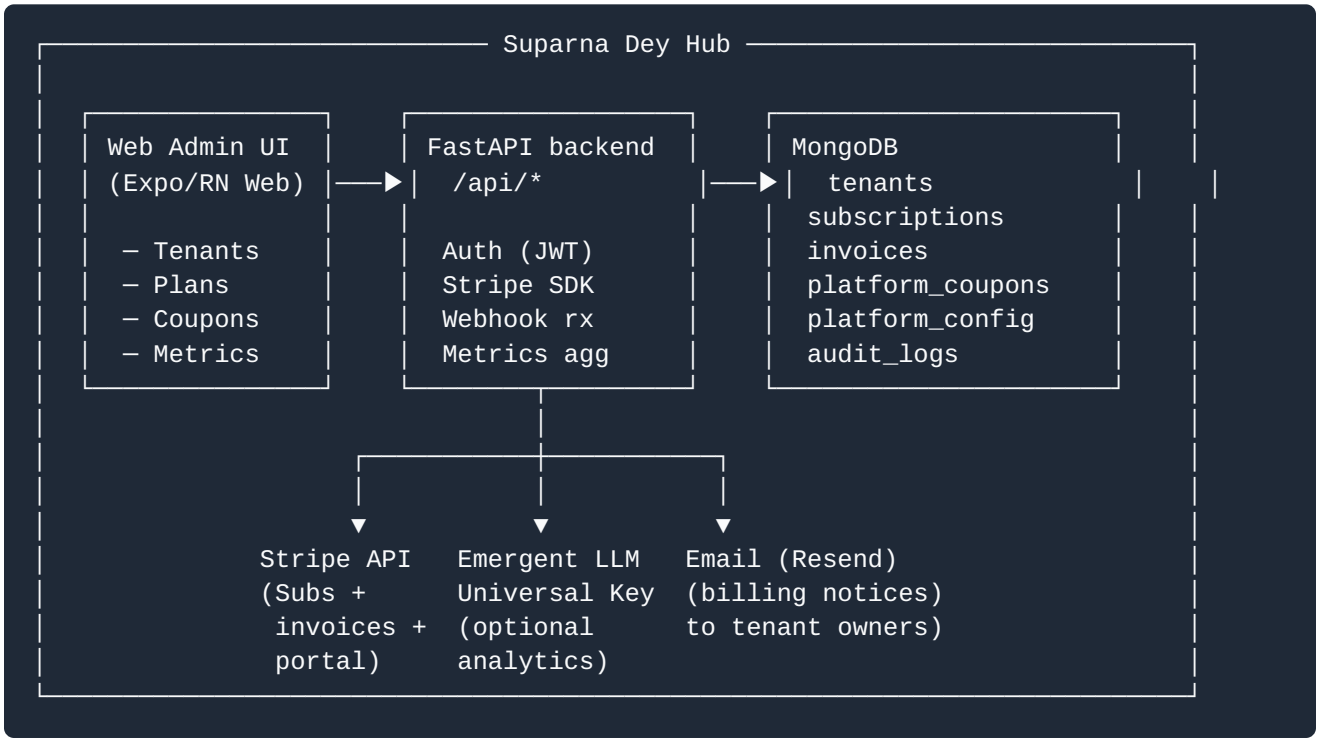
The tradeoff is the hub has to *know* about tenants (registry) and tenants have to *phone home* (billing). Both are cheap: a JSON registry + a 5-minute poller.

3 · Hub — Suparna Dey Super Admin

3.1 · Purpose

The hub is the **single seat of billing truth** and **the tenant directory** for the entire platform. It never handles customer orders, menus, or delivery — those live entirely on each tenant fork.

3.2 · Internal composition



3.3 · Public API surface (called by tenant spokes)

Endpoint	Purpose
`GET /api/tenant/status`	The 5-min phone-home check
`GET /api/tenant/billing/summary`	Plan, next-billing, card, invoice history
`POST /api/tenant/billing/checkout`	Returns a Stripe Checkout URL
`POST /api/tenant/billing/portal`	Returns a Stripe Customer Portal URL
`POST /webhooks/stripe`	Stripe → hub: sub state transitions
`GET /api/tenant/platform-coupons`	Phase 4: available discounts

All endpoints require `X-Tenant-Id` + `X-Tenant-Api-Key` headers.

3.4 · Internal API surface (super-admin UI only, JWT-guarded)

Endpoint	Purpose
`POST /api/admin/tenants`	Create a new tenant → issues API key
`PATCH /api/admin/tenants/{tid}`	Change plan, suspend, rotate api_key
`POST /api/admin/plans`	Create/edit Stripe products & prices
`POST /api/admin/coupons`	Create a platform-wide coupon
`GET /api/admin/metrics/*`	Global usage, revenue, LLM spend

4 · Spoke — a tenant fork

A tenant is a **complete restaurant SaaS** running in its own Emergent project. Nothing about the tenant assumes the hub is up — if the hub goes down, the tenant continues serving orders (fail-open on the last known-good subscription state).

4.1 · Repository layout

```
/app
├── brand.json                        ← identity + region + platform creds
├── brand.config.js -> frontend/      ← convenience symlink
├── frontend/
│   ├── brand.config.js               ← design tokens + copy
│   ├── app/                          ← expo-router routes
│   │   ├── (customer)/               ← customer app screens
│   │   ├── admin/                    ← admin web screens
│   │   └── delivery-partner.tsx      ← driver mobile screen
│   ├── src/
│   │   ├── components/               ← shared UI (Bag, Map, Banner, …)
│   │   ├── screens/                  ← feature composites
│   │   ├── theme.ts / brand.ts       ← consumes brand.config.js
│   │   ├── api.ts / adminApi.ts      ← HTTP clients
│   │   └── utils/                    ← storage, comboExpand, backendUrl…
│   └── assets/brand/                 ← logo / splash / icons per tenant
├── backend/
│   ├── server.py                     ← monolithic FastAPI (Phase 1-3)
│   ├── brand.py                      ← reads brand.json
│   ├── platform_client.py            ← the 5-min phone-home poller
│   ├── billing_client.py             ← on-demand hub billing calls
│   ├── tools/
│   │   ├── seed_export.py
│   │   ├── seed_import.py
│   │   └── seed_emergency_orders.py ← ad-hoc test seeders
│   └── web_dist/                     ← compiled Expo web SPA (served by FastAPI)
├── docs/
│   ├── ops/                          ← runbooks, intake, tests, this doc
│   ├── HUB_BILLING_SPEC.md           ← handoff spec for the hub project
│   └── seed-packs/                   ← menu templates
└── scripts/
    ├── scrub_for_template.py         ← run before pushing to GitHub template
    ├── generate_tenant_secrets.py    ← one-shot env-var generator
    └── build_docs.py                 ← markdown → PDF
```

4.2 · Tenant runtime — component view



4.3 · Config-driven branding

Everything the customer sees (name, colors, logo, copy, currency, timezone, contact links, bundle IDs, deep-link schemes) is driven by two files:

File	Owns	Consumed by
`brand.json`	Identity + platform creds	Backend (`brand.py`)
`frontend/brand.config.js`	Design tokens + native ids + copy	Frontend (`brand.ts` , `theme.ts`)

The backend and frontend each have a single import layer that reads these files once at startup. **No feature code references the tenant name literally.** This is what makes a 30-minute rebrand possible.

---

5 · The three apps per tenant

All three apps ship from the **same Expo codebase**, differentiated by route. Two of them (customer, driver) also build into native `.ipa` / `.apk` bundles with their own bundle IDs.

5.1 · Customer mobile app

High-level

Capability	Present
Browse this week's menu	
Browse upcoming weeks & history	
Item detail + photo	
Cart with quantity + combo builder	
Weekly-combo picker (X items)	
Catering packages (tiered, choice groups, premium items)	
Google Maps address autocomplete	
Delivery slot picker	
Coupon codes	
Loyalty points earn + redeem	(redeem Phase 4)
Payment: COD, Interac	
Payment: Stripe card	(temporarily stubbed)
Payment: PayPal	🗺 Roadmap
Order confirmation email	
Live driver tracking (map + ETA)	
Push notification on status change	(native builds only)
Google Sign-In	
Guest checkout	
Save & manage addresses	
Delete account (App Store req'd)	

Detailed feature list

Home / discovery - Current-week hero banner with tenant tagline - Featured items rail (admin-flagged) - Category filters (veg / non-veg / desserts / sides) - Search across current-week items - "Upcoming" tab for peeking at next weeks - "History" tab for re-ordering past favorites

Cart / bag - Persisted via AsyncStorage (survives app kill) - Item quantity +/-, swipe-to-remove - Combo builder: pick N items from an eligible pool with optional per-item surcharge - Live subtotal, delivery-fee, tax preview - "Add $X more to check out" nudge if below minimum

Checkout - Google Places autocomplete (Canada-restricted) - Manual-entry fallback - Google Places details → auto-fill line1 / city / postal / lat / lng - Delivery slot: filtered to admin-configured delivery days - Cut-off enforcement (e.g. "orders close Wed 8 pm for this weekend") - Coupon field with instant validation - Loyalty points slider (Phase 4) - Payment method radio (COD / Interac / Card) - Special instructions free-text - Server-side quote → order create (two-step to prevent price drift)

Order tracking - Timeline: placed → confirmed → preparing → out for delivery → delivered - Live driver dot on Google Map once driver accepts - Ticking ETA (recomputed every 30 s from driver position) - "Your driver has arrived" banner - Delivered → prompt for tip / review

Account - Google Sign-In (Emergent-managed OAuth relay) - Guest mode with 7-day session - Profile: name, email, phone, default address - Addresses list (up to N) - Loyalty balance + earn history - Push token registration - Delete-account with grace warning (branded with tenant name)

5.2 · Delivery partner mobile app

High-level

Capability	Present
Password login (per-tenant driver)	
Live GPS location broadcasting	
Active orders queue	
Order detail with map + directions	
Status transitions (picked / on-way / arrived / delivered)	
History of past deliveries	
Earnings (per-order + weekly)	
Deactivation by admin blocks login	
Push on new assignment	(native builds only)
Route optimization	🗺 Roadmap
Delivery-proof photo + signature	🗺 Roadmap

Detailed feature list

Login - Email + password (bcrypt-hashed on server) - Rate-limited by IP - SaaS gate: 402 if tenant's subscription is suspended

Live location - `expo-location watchPositionAsync` at 5-second cadence when app foreground - Background location with proper permission handshake (native only) - Server persists last known lat/lng on `delivery_partners` doc - Also stamps the driver's active `orders` for the customer's live map

Order queue - Filtered to `delivery_partner_id = <me>` + `delivery_status ∈ active` - Sorted by delivery slot start time - Pull-to-refresh

Order actions - **Accept** → status: `picked_up` - **On the way** → status: `on_the_way` (triggers customer's ticking ETA) - **Arrived** → status: `arrived` (customer sees arrival banner) - **Delivered** → status: `delivered` (loyalty points credited to customer)

History & earnings - Delivered orders list, day/week filter - Totals: number of drops, distance covered, tip totals

5.3 · Admin web app

Served via Expo Web from `backend/web_dist/index.html`. Runs in any modern desktop browser.

High-level

Capability	Present
JWT-based admin login	
Dashboard with revenue / orders / customers	
Master menu CRUD	
Weekly menu CRUD + publish scheduling	
Weekly-combo package configuration	
Catering package builder (tiers, groups, premium)	
Orders list, filters, refunds	
Admin-created "New Order" workflow	
Customer management + loyalty credit	
Delivery partner invite + assign	
Business settings (hours, fees, tax, delivery areas)	
Billing tab (subscribe / manage / invoices)	Phase 3
Platform banner when SaaS suspended	
Promotions / coupon manager for customers	(partial)
Reviews moderation	🗺 Roadmap
Reports (P&L, tax exports)	🗺 Roadmap

Detailed feature list

Dashboard - Gross revenue (all-time + this month + today) - All-orders / active / delivered / customers counters - Recent activity feed

Master menu - Add/edit/delete/duplicate items - Categories, cuisine, veg-type, spice level, allergens - Cost price (super-admin visibility only) - Image upload (Emergent Object Storage) + Gemini Nano Banana AI generation - Bulk publish / unpublish

Weekly menu - Publish current + up to 12 future weeks - Each week: pick items from Master → line-level price override - Enable weekly-combo package (X items for flat price + surcharges) - Stock limits per line (auto sold-out when consumed)

Catering - Package builder with per-person price OR upgrade tiers - Choice groups with min/max select - Premium items with per-person upcharge - Custom-quote mode (contact form) for wedding-scale events

Orders - Filters: status, payment method, date range, driver - Detail view: full item snapshot, address (map), timeline - Refund flow (records refund without hitting Stripe when Stripe stubbed) - Print / share receipt

New Order (admin creates on behalf of customer) - Customer picker (search by name/email/phone) - Item picker with weekly-combo builder - Address entry + delivery slot - Optional driver assignment → auto-status "packed"

Customers - Full list with created_at, order count, lifetime value - Detail: profile, addresses, order history, loyalty history - Credit loyalty points

Delivery - Invite driver: name, email, phone, vehicle, password - Deactivate (soft delete — unassigns live orders) - Assign order to specific driver - See each driver's current location & active queue

Settings - Business name / phone / email / kitchen address - Operating hours (structured 0=Sun … 6=Sat) - Ordering days vs delivery days - Delivery areas (autocomplete filter) - Default delivery fee (cents) - Minimum order value - Tax config: enabled, rate (basis points), label - Per-payment-method config (enabled + tax) for stripe/cod/interac/paypal

Billing (Phase 3) - Subscribe / manage / invoice history - Stripe Checkout (hub-hosted) - Stripe Customer Portal (hub-hosted) - Real-time status: active / trialing / past_due / canceled / unpaid - Trial-end date if applicable - Cache: 60 s server-side, force-refresh via pull-to-refresh

Platform banner - Auto-shows when tenant's subscription is not `active` - Copy explicitly names the *restaurant owner* as the party who missed billing (so customers don't feel blamed) - CTA routes directly to the Billing tab

---

6 · Cross-cutting concerns

6.1 · Authentication

Three distinct auth systems live side-by-side in every tenant:

```
┌──────────────────┬──────────────────┬──────────────────────────┐
│ Actor            │ Mechanism        │ Token lifetime           │
├──────────────────┼──────────────────┼──────────────────────────┤
│ Customer (real)  │ Google OAuth via │ 7-day session token      │
│                  │ Emergent relay   │ (`user_sessions` coll)   │
│ Customer (guest) │ Silent /auth/    │ 7-day session token      │
│                  │ guest endpoint   │                          │
│ Admin            │ Email + password │ JWT HS256, 24 h          │
│                  │ (bcrypt hashed)  │                          │
│ Delivery driver  │ Email + password │ JWT HS256, 12 h          │
│                  │ (bcrypt hashed)  │                          │
└──────────────────┴──────────────────┴──────────────────────────┘
```

6.2 · Payments

Currently active on every tenant:

Method	Enabled	Flow
COD		Order → confirmed. Driver collects cash on delivery.
Interac		Order → confirmed. Instructions shown to customer.
Stripe	⏸	Stubbed w/ 503; endpoints preserved for Phase 4 re-enable
PayPal	🗺 Roadmap	Config-ready, endpoints not shipped

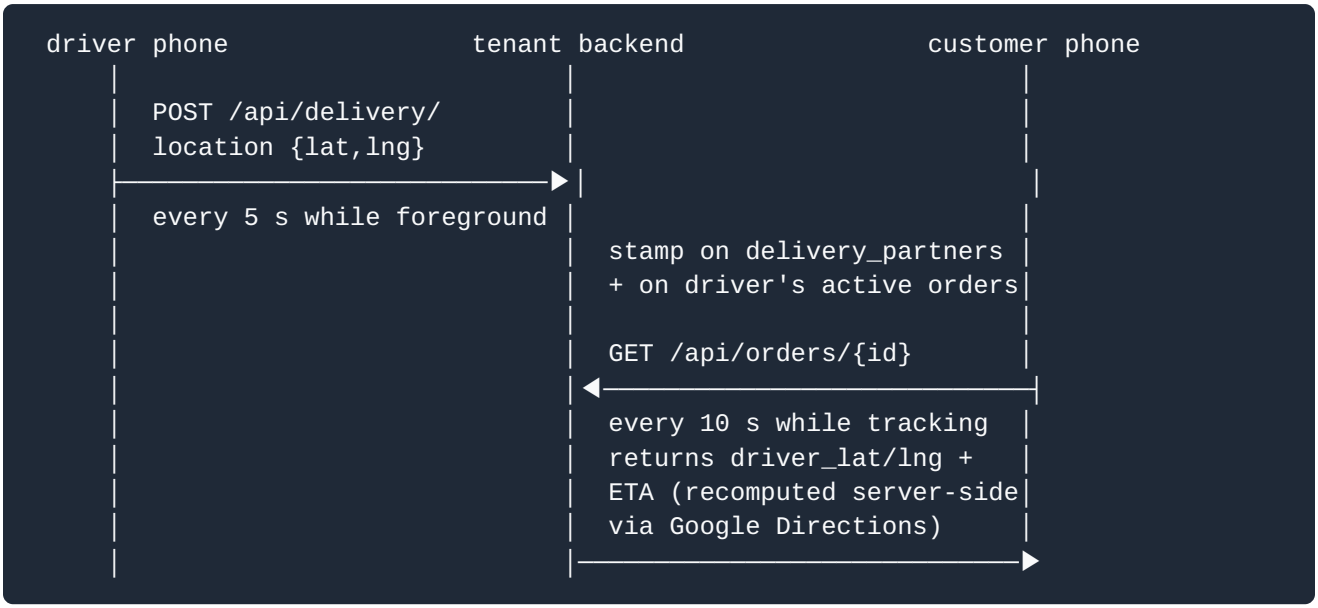
Platform SaaS billing (tenant → hub) *does* use Stripe — but that Stripe integration lives on the hub, not on tenants.

6.3 · Notifications

Channel	Provider	Use
Push	Emergent Push	Order status, driver assignment
Email	Emergent Email (Resend)	Order confirmations, receipts, deletion
SMS	Twilio	OTP login, delivery arrival (optional)
In-app	Local	Banner + toast for cart/coupon/etc

Push notifications **only work on native builds** (TestFlight / Play Store). Expo Go and the web preview cannot receive them.

6.4 · Live tracking



7 · Data model

Every tenant's MongoDB has the same collections. Nothing tenant-specific is in the schema — the *value* of every document belongs to that tenant's restaurant.

7.1 · Collections summary

Collection	Purpose	~doc count/mo (avg)
`users`	Customers + guests + admin + drivers	~500
`user_sessions`	Customer bearer tokens	~500
`master_menu`	Reusable dish templates	~50 (stable)
`weekly_menus`	Per-week menu with combo config	~4/mo
`catering_packages`	Catering options	~5 (stable)
`orders`	Every customer purchase	~1,000
`payment_transactions`	Stripe session bookkeeping (dormant)	0 (stubbed)
`coupons`	Discount codes	~10 (stable)
`delivery_partners`	Driver roster	~5 (stable)
`push_tokens`	Device push tokens	~500
`business_settings`	One doc, keyed `default`	1
`platform_status`	Cached SaaS subscription state (from hub)	1
`catering_quote_requests`	Custom-quote lead pipeline	~5/mo

7.2 · Key document shapes

Order (abbreviated):

```
{
  "order_id": "ord_a1b2c3d4",
  "user_id": "user_xyz",
  "items_snapshot": [ { "kind": "weekly_package", "ref_id": "menu_xxx", ... } ],
  "address": { "line1": "…", "city": "…", "postal_code": "…", "lat": 43.6, "lng": -79.6 },
  "slot":    { "date": "2026-09-05", "window": "saturday_morning" },
  "subtotal_cents": 8900, "delivery_fee_cents": 0, "tax_cents": 1157, "total_cents": 10057,
  "payment_method": "cod", "status": "out_for_delivery", "payment_status": "cod_pending",
  "delivery_partner_id": "dp_abc", "delivery_status": "assigned",
  "delivery_partner_lat": 43.5890, "delivery_partner_lng": -79.6441,
  "tracking_events": [ … ]
}
```

Weekly menu:

```
{
  "menu_id": "menu_xxx", "week_start": "2026-08-23", "title": "This Week's Bengali Table",
  "published": true, "status": "published",
  "package_enabled": true,
  "package": { "name": "New Combo", "price_cents": 8900, "pick_count": 6,
               "eligible_master_ids": [], "item_surcharges_cents": { "mi_mutton": 500 } },
  "lines": [ { "master_item_id": "mi_abc", "price_override_cents": null,
               "availability": "available", "stock_used": 0 } ]
}
```

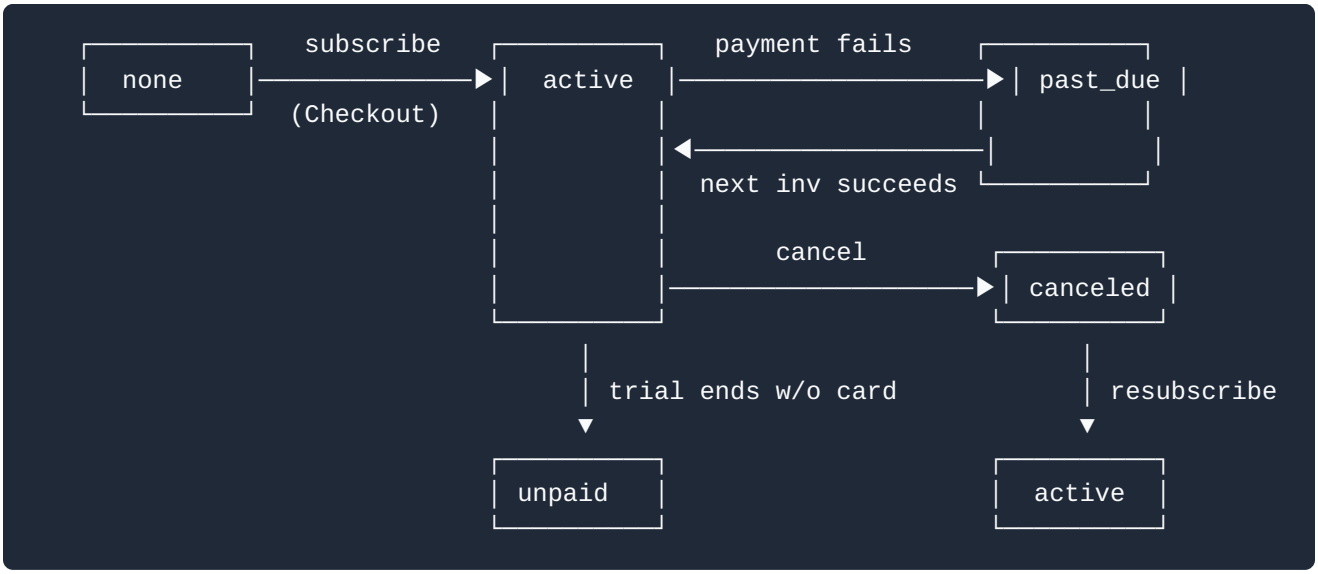
8 · Integration ecosystem



Every integration on this tenant has a **graceful-degrade** path: - Google Maps down → manual address entry still works - Emergent Email down → order still created, admin sees a warning - Twilio down → skip SMS, don't fail login - Emergent LLM down → menu descriptions fall back to admin-typed text - Push down → in-app notification still shows on next foreground

9 · SaaS lifecycle

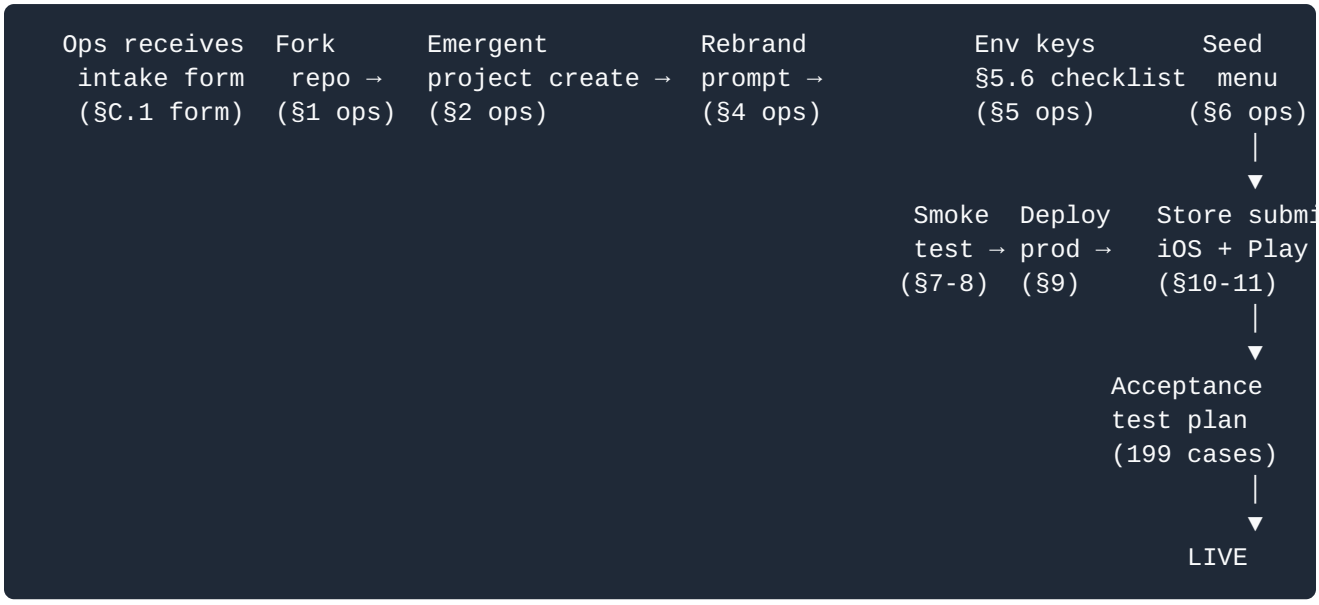
9.1 · Tenant states (from the hub's perspective)



9.2 · Tenant-side effect of each state

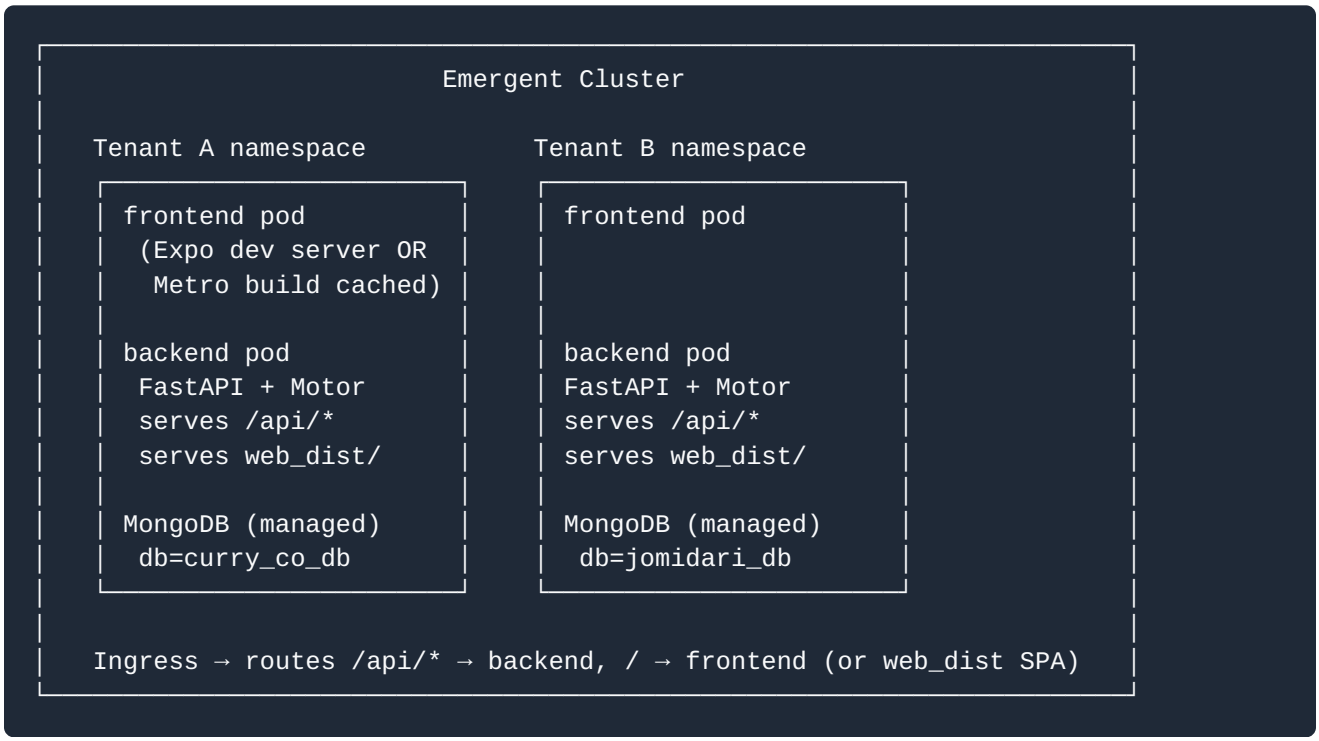
State	Customer checkout	Driver login	Admin portal	Banner
`active`				none
`trialing`				"Trial ends [date]"
`past_due`	402	402	(with banner)	red banner
`unpaid`	402	402	(with banner)	red banner
`canceled`	402	402	(with banner)	red banner + resubscribe CTA
`unconfigured`	(fail-open)			none

9.3 · Onboarding pipeline



10 · Deployment architecture

10.1 · Per-tenant deployment on Emergent



10.2 · Web bundle handling

- **Preview** (Emergent dev): Metro serves everything from `/app/frontend`.
- **Production** (Emergent Publish): `npx expo export -p web` builds a static SPA into `backend/web_dist/`. FastAPI then serves `web_dist/index.html` on any non-`/api/*` route (SPA fallback).

This means the admin portal keeps working even when Metro isn't running — critical for K8s pods that restart frequently.

10.3 · Native builds

Emergent's Publish flow builds: - **iOS**: `.ipa` signed with the tenant's Apple Team ID - **Android**: `.apk` (for sideload testing) + `.aab` (for Play Store)

Bundle IDs come from `native.customer.bundleIdentifier` and `native.delivery.bundleIdentifier` in `brand.config.js`.

11 · Security model

11.1 · Boundaries

Trust zone	Contents
Public internet	Customer + driver mobile apps, admin browser
Emergent ingress	HTTPS termination, request routing
Tenant backend pod	FastAPI, .env secrets, LLM/Email keys
Tenant MongoDB	Isolated per tenant, network policy locks
Hub	Stripe keys, tenant registry — HIGH VALUE

11.2 · Key hygiene

- No secret is ever committed to git. `scripts/scrub_for_template.py` blanks tenant creds before pushing to the shared template.
- `generate_tenant_secrets.py` produces fresh JWT + Fernet + admin passwords per tenant on onboarding.
- Google Maps key is **shared** across tenants **but restricted** per-tenant to specific bundle IDs + HTTP referrers.
- Twilio credentials are shared account-level; per-tenant isolation is by Messaging Service SID.
- Emergent LLM / Email / Push keys are per-Emergent-account.

11.3 · Auth threat model

- **Admin brute force** — rate-limited at the endpoint level, JWT-based.
- **Guest session hijack** — 7-day expiry, per-request signature check.
- **Driver credential leak** — deactivating a driver in Admin immediately invalidates their token via `is_active` check on every request.
- **Hub API key leak** — hub revokes + rotates `api_key`, next 5-min poll flips tenant into `unauthorized` banner.

- **CORS** — locked to the tenant's deployed origins.

11.4 · Data lifecycle

- **PII in orders**: name, email, phone, address, lat/lng.
- **Deletion**: `DELETE /api/user/me` anonymises historical orders (keeps financial records for audit) and hard-deletes the user + sessions + push tokens.
- **Retention**: no automatic deletion. Ops manually purges canceled tenants ~90 days after cancellation.

12 · Scalability & operations

12.1 · Where the platform scales linearly

Dimension	Bottleneck	Notes
More tenants	Emergent cluster capacity	Each tenant is a full pod set; scale = spin up more nodes
More customers per tenant	MongoDB IOPS on that tenant	Isolated per fork; noisy neighbour risk is zero
Orders per second per tenant	FastAPI CPU + Mongo write	Vertical scale first, then horizontal (uvicorn workers)
Google Maps quota	$200/mo free credit per project	Shared project; per-tenant metering not tracked yet
LLM spend	Emergent Universal Key budget	Per-Emergent-account; ops tops up as needed

12.2 · Observability

Every tenant emits: - **Backend logs** → Emergent dashboard (retained ~7 days) - **Frontend errors** → in-app error boundary + logs - **Platform status** → `/api/platform/status` endpoint for health probes - **Ops metrics (hub side)** → per-tenant order volume, revenue, MRR

12.3 · Runbook links

- `docs/ops/TENANT_ONBOARDING_OPS.md` — spinning up a tenant
- `docs/ops/TENANT_ACCEPTANCE_TESTS.md` — 199-case pre-launch checklist
- `docs/HUB_BILLING_SPEC.md` — the hub contract this doc references
- `docs/PUBLISH_SOP.md` — App Store / Play Store publishing SOP

13 · Roadmap

Phase	Scope	Status
1	Multi-brand template — `brand.json` + `brand.config.js` drive everything	Done
2	Phone-home poller — tenant reads subscription status from hub every 5 min, gates checkout + driver login	Done
3	Tenant Billing tab — subscribe / manage / invoices via hub-owned Stripe	Done
4	Platform-wide coupons — super admin issues SaaS discounts / trials; tenants redeem	Next
5	Admin Wave 2 — reviews moderation, customer promotions manager, reports	🗺
6	PayPal + crypto checkout for customers	🗺
7	Delivery-proof photo + signature capture	🗺
8	Route optimization on driver app	🗺
9	Multi-lingual customer app (Bengali, Hindi, French)	🗺
10	Reviews & ratings pipeline end-to-end	🗺

Appendix A · Glossary

Term	Definition
Hub	The Suparna Dey Platform Super Admin. Single deployment, single Stripe.
Spoke	A tenant fork. Own deployment, own DB, own brand.
Tenant	A specific restaurant running a fork.
Fork	A copy of `Curry-Co-Template` on GitHub, rebranded and deployed.
Template	The repository (`Curry-Co-Template`) that every tenant is forked from.
Weekly package	A "pick N items for flat price" combo scoped to one week's menu.
Master menu	Permanent library of dishes; weekly menus reference these.
Phone-home	The tenant backend's 5-minute poll to the hub's status endpoint.
Universal Key	Emergent's shared LLM key that works across OpenAI / Anthropic / Google.

Appendix B · What lives where (cheat sheet)

Thing	Repo	Deployment
Customer mobile UI	tenant fork	Emergent project (tenant)
Driver mobile UI	tenant fork	Emergent project (tenant)
Admin web UI	tenant fork	Emergent project (tenant)
Order/menu/catering logic	tenant fork	Emergent project (tenant)
Stripe integration	hub	Emergent project (hub)
Tenant registry	hub	Emergent project (hub)
Platform coupons	hub	Emergent project (hub)
Google Cloud project	(external)	Google Cloud
Twilio account	(external)	Twilio
App Store / Play listings	(external)	Apple / Google